



北京師範大學
BEIJING NORMAL UNIVERSITY

高维逻辑回归的惩罚估计算法

郑泽靖 202331011011

2024 年 8 月 29 日

目录

1	高维逻辑回归的惩罚估计算法	3
1.1	逻辑回归	3
1.2	高维逻辑回归 LASSO 算法	4
1.2.1	高维逻辑回归 LASSO 的 Newton-Raphson 算法	4
1.2.2	高维逻辑回归 LASSO 的 ADMM 算法	5
1.3	高维逻辑回归的岭回归	6
1.4	高维逻辑回归自适应 LASSO	6
1.5	高维逻辑回归的弹性网模型	6
1.6	高维逻辑回归 SCAD	6
1.6.1	LQA 算法	7
1.6.2	MM 算法	7
1.6.3	LLA 算法	8
2	模拟	8
2.1	模拟设置	8
2.2	数值结果	9
2.3	ROC 曲线图	12

1 高维逻辑回归的惩罚估计算法

1.1 逻辑回归

逻辑回归 (Logistic Regression) 是一种广泛应用于统计学和机器学习的分类算法, 主要用于二分类问题。它的基本思想是利用逻辑函数 (sigmoid 函数) 来将模型的线性组合映射到概率值, 从而进行分类。其模型公式为:

$$P(y = 1|\mathbf{x}) = \sigma(\mathbf{x}^\top \boldsymbol{\beta}) = \frac{1}{1 + e^{-\mathbf{x}^\top \boldsymbol{\beta}}}$$

其中 $\mathbf{x} \in \mathbb{R}^p$ 为协变量, $\boldsymbol{\beta} \in \mathbb{R}^p$ 为参数向量, $\sigma(z)$ 是 sigmoid 函数。对应地, 类别 $y = 0$ 的概率为:

$$P(y = 0|\mathbf{x}; \boldsymbol{\beta}) = 1 - P(y = 1|\mathbf{x}; \boldsymbol{\beta}) = 1 - \sigma(\mathbf{x}^\top \boldsymbol{\beta})$$

对于一组训练数据 $(\mathbf{x}_i, y_i)$ (其中 $i = 1, 2, \dots, n$), 逻辑回归模型的似然函数 $L_n(\boldsymbol{\beta})$ 可以表示为:

$$L_n(\boldsymbol{\beta}) = \prod_{i=1}^n P(y_i|\mathbf{x}_i; \boldsymbol{\beta})$$

根据我们对 y_i 的概率假设, 可以将似然函数写成:

$$L_n(\boldsymbol{\beta}) = \prod_{i=1}^n \sigma(\mathbf{x}_i^\top \boldsymbol{\beta})^{y_i} \cdot (1 - \sigma(\mathbf{x}_i^\top \boldsymbol{\beta}))^{(1-y_i)}$$

为了简化计算, 通常我们取对数, 得到对数似然函数 $l_n(\boldsymbol{\beta})$:

$$l_n(\boldsymbol{\beta}) = \log L_n(\boldsymbol{\beta}) = \sum_{i=1}^n [y_i \log(\sigma(\mathbf{x}_i^\top \boldsymbol{\beta})) + (1 - y_i) \log(1 - \sigma(\mathbf{x}_i^\top \boldsymbol{\beta}))]$$

负对数似然损失函数可以写为:

$$J_n(\boldsymbol{\beta}) = - \sum_{i=1}^n [y_i \log(\sigma(\mathbf{x}_i^\top \boldsymbol{\beta})) + (1 - y_i) \log(1 - \sigma(\mathbf{x}_i^\top \boldsymbol{\beta}))]$$

在优化算法中, 我们可能需要负对数似然损失函数的梯度和 Hessian 矩阵, 下面我们进行推导。我们先计算该损失函数的梯度 $\nabla J_n(\boldsymbol{\beta})$ 。使用链式法则, 我们可以得到:

$$\nabla J_n(\boldsymbol{\beta}) = - \sum_{i=1}^n \left[y_i \frac{1}{\sigma(\mathbf{x}_i^\top \boldsymbol{\beta})} \nabla \sigma(\mathbf{x}_i^\top \boldsymbol{\beta}) + (1 - y_i) \frac{-1}{1 - \sigma(\mathbf{x}_i^\top \boldsymbol{\beta})} \nabla \sigma(\mathbf{x}_i^\top \boldsymbol{\beta}) \right]$$

对于 sigmoid 函数 $\sigma(z)$, 其导数为:

$$\nabla \sigma(z) = \sigma(z)(1 - \sigma(z))$$

因此, $\nabla \sigma(\mathbf{x}_i^\top \boldsymbol{\beta})$ 可以表示为:

$$\nabla \sigma(\mathbf{x}_i^\top \boldsymbol{\beta}) = \sigma(\mathbf{x}_i^\top \boldsymbol{\beta}) (1 - \sigma(\mathbf{x}_i^\top \boldsymbol{\beta})) \mathbf{x}_i$$

将这个结果代入梯度公式，我们可以得到：

$$\nabla J_n(\boldsymbol{\beta}) = - \sum_{i=1}^n \left[y_i \frac{\sigma(\mathbf{x}_i^\top \boldsymbol{\beta}) (1 - \sigma(\mathbf{x}_i^\top \boldsymbol{\beta}))}{\sigma(\mathbf{x}_i^\top \boldsymbol{\beta})} \mathbf{x}_i - (1 - y_i) \frac{\sigma(\mathbf{x}_i^\top \boldsymbol{\beta}) (1 - \sigma(\mathbf{x}_i^\top \boldsymbol{\beta}))}{1 - \sigma(\mathbf{x}_i^\top \boldsymbol{\beta})} \mathbf{x}_i \right]$$

简化后得到：

$$\nabla J_n(\boldsymbol{\beta}) = \sum_{i=1}^n (\sigma(\mathbf{x}_i^\top \boldsymbol{\beta}) - y_i) \mathbf{x}_i$$

Hessian 矩阵是梯度的梯度，即：

$$H_n(\boldsymbol{\beta}) = \nabla^2 J_n(\boldsymbol{\beta}) = \nabla \left(\sum_{i=1}^n (\sigma(\mathbf{x}_i^\top \boldsymbol{\beta}) - y_i) \mathbf{x}_i \right) = \sum_{i=1}^n \sigma(\mathbf{x}_i^\top \boldsymbol{\beta}) (1 - \sigma(\mathbf{x}_i^\top \boldsymbol{\beta})) \mathbf{x}_i \mathbf{x}_i^\top$$

关于逻辑回归的更多内容可以参考 [1, 2]。

1.2 高维逻辑回归 LASSO 算法

在高维统计的框架下， $p \gg n$ ，MLE 的有效性和可靠性可能受到显著影响。高维逻辑回归中的 LASSO (Least Absolute Shrinkage and Selection Operator) 算法是一种通过 L1 正则化来实现特征选择和模型稀疏的有效方法 [3]。LASSO 逻辑回归的基本目标是最小化以下损失函数：

$$\min_{\boldsymbol{\beta}} J_n(\boldsymbol{\beta}) + \lambda \sum_{j=1}^p |\beta_j|$$

其中 λ 是正则化参数，控制 LASSO 的稀疏性。由于 L1 正则化的非光滑性，LASSO 的直接优化较为困难。在 R 语言中，可以使用 `glmnet` 函数进行 LASSO 估计。下面我们两种 LASSO 估计的优化算法。

1.2.1 高维逻辑回归 LASSO 的 Newton-Raphson 算法

Newton-Raphson 算法是一种用于求解非线性方程的迭代方法。对于逻辑回归，给定初值 $\boldsymbol{\beta}^{(0)}$ 后，它通过以下步骤更新参数：

$$\boldsymbol{\beta}^{(t+1)} = \boldsymbol{\beta}^{(t)} - H_n(\boldsymbol{\beta}^{(t)})^{-1} g_n(\boldsymbol{\beta}^{(t)})$$

对于 LASSO 逻辑回归，由于 L1 范数的非光滑性，LASSO 目标函数的 L1 正则化部分在 $\beta_j = 0$ 时不可微，使得传统的 Newton-Raphson 方法不适用。由于 LASSO 的非光滑性，通常采用次梯度法 (sub-gradient method) 来处理。L1 范数的次梯度 $\mathbf{S}$ 为

$$S_j = \begin{cases} 1 & \text{if } \beta_j > 0 \\ -1 & \text{if } \beta_j < 0 \\ \text{任意值} & \text{if } \beta_j = 0 \end{cases}$$

最终的迭代公式为

$$\boldsymbol{\beta}^{(t+1)} = \boldsymbol{\beta}^{(t)} - \alpha(H_n(\boldsymbol{\beta}^{(t)})^{-1}g_n(\boldsymbol{\beta}^{(t)}) + \lambda\mathbf{S})$$

其中 α 是学习率。

1.2.2 高维逻辑回归 LASSO 的 ADMM 算法

在高维逻辑回归中，LASSO 可以通过交替方向乘子法（ADMM, Alternating Direction Method of Multipliers）进行高效优化。ADMM 将原始问题转化为带有拉格朗日乘子的约束问题。我们可以将 LASSO 的 L1 正则化项引入为一个约束条件。具体步骤如下：

1. 引入辅助变量使得 $\mathbf{z} = \boldsymbol{\beta}$ ，现在我们要最小化以下目标，

$$\min_{\mathbf{z}, \boldsymbol{\beta}} J_n(\boldsymbol{\beta}) + \lambda \|\mathbf{z}\|_1, \text{ subject to } \mathbf{z} = \boldsymbol{\beta}$$

2. 构造增广的拉格朗日函数：

$$\mathcal{L}_\eta(\boldsymbol{\beta}, \mathbf{z}, \mathbf{u}) = J_n(\boldsymbol{\beta}) + \lambda \|\mathbf{z}\|_1 - \mathbf{u}^\top (\mathbf{z} - \boldsymbol{\beta}) + \frac{\eta}{2} \|\mathbf{z} - \boldsymbol{\beta}\|_2^2$$

其中 $\mathbf{u}$ 是拉格朗日乘子， η 是一个固定的正常数。

3. ADMM 的更新步骤

- (a) 更新 $\boldsymbol{\beta}$:

$$\boldsymbol{\beta}^{(t+1)} = \arg \min_{\boldsymbol{\beta}} J_n(\boldsymbol{\beta}) - (\mathbf{u}^{(t)})^\top (\mathbf{z}^{(t)} - \boldsymbol{\beta}) + \frac{\eta}{2} \|\mathbf{z}^{(t)} - \boldsymbol{\beta}\|_2^2$$

可以利用梯度下降算法求解。

- (b) 更新 $\mathbf{z}$:

$$\begin{aligned} \mathbf{z}^{(t+1)} &= \arg \min_{\mathbf{z}} \lambda \|\mathbf{z}\|_1 - (\mathbf{u}^{(t)})^\top (\mathbf{z} - \boldsymbol{\beta}^{(t+1)}) + \frac{\eta}{2} \|\mathbf{z} - \boldsymbol{\beta}^{(t+1)}\|_2^2 \\ \mathbf{z}_j^{(t+1)} &= \text{sign}(\boldsymbol{\beta}_j^{(t+1)} + \frac{\mathbf{u}_j^{(t)}}{\eta}) \max\{|\boldsymbol{\beta}_j^{(t+1)} + \frac{\mathbf{u}_j^{(t)}}{\eta}| - \frac{\lambda}{\eta}, 0\} \end{aligned}$$

$j \in 1, \dots, p$, 这里 $\text{sign}()$ 为符号函数。

- (c) 更新拉格朗日乘子 $\mathbf{u}$

$$\mathbf{u}^{(k+1)} = \mathbf{u}^{(k)} + \boldsymbol{\beta}^{(k+1)} - \mathbf{z}^{(k+1)}$$

在模拟中， λ 使用 `cv.glmnet` 函数进行选择， $\eta = 1$ ， $\mathbf{u}$ 的初值为单位向量 $\mathbf{1}_p$ 。 $\boldsymbol{\beta}$ 和 $\mathbf{z}$ 的初值为极大似然估计的结果，使用函数 `glm`。

1.3 高维逻辑回归的岭回归

岭回归 (Ridge Regression) 是一种用于处理模型中多重共线性和过拟合问题的正则化方法。岭回归通过引入 L2 正则化项来约束模型参数，从而提高模型的泛化能力。岭回归的目标函数可以表示为：

$$\min_{\beta} J_n(\beta) + \lambda \|\beta\|_2$$

岭回归的损失函数是凸的，通常使用梯度下降或牛顿法等优化算法来求解。在 R 语言中，使用 `glmnet` 函数进行拟合。

1.4 高维逻辑回归自适应 LASSO

自适应 LASSO (Adaptive Lasso) [4] 是一种通过引入自适应权重的方式来改进 LASSO 回归的特征选择性能的方法。自适应 LASSO 的目标是最小化以下损失函数：

$$\min_{\beta} J_n(\beta) + \lambda \sum_{j=1}^p w_j |\beta_j|$$

其中 w_j 是自适应权重，通常根据初始估计的参数来计算。自适应权重 w_j 的计算通常基于初始模型的系数。一般步骤如下：

1. **初步估计**：使用普通 LASSO 或其他方法得到初步的参数估计 $\hat{\beta}^{(0)}$ 。
2. **计算权重**：

$$w_j = \frac{1}{|\hat{\beta}_j^{(0)}| + \epsilon}$$

其中 ϵ 是一个小常数，用于避免除以零的情况。

1.5 高维逻辑回归的弹性网模型

弹性网 (Elastic Net) [5] 是一种结合了 LASSO (L1 正则化) 和岭回归 (L2 正则化) 优点的方法。弹性网通过同时使用 L1 和 L2 正则化，从而实现特征选择和参数收缩。弹性网的目标是最小化以下损失函数：

$$\min_{\beta} J_n(\beta) + \lambda \alpha \|\beta\|_1 + \frac{\lambda(1-\alpha)}{2} \|\beta\|_2^2$$

其中参数 α 决定了 L1 惩罚的强度，当 $\alpha = 1$ 时弹性网方法等价于 LASSO，而当 $\alpha = 0$ 时弹性网方法等价于岭回归。

1.6 高维逻辑回归 SCAD

SCAD (Smoothly Clipped Absolute Deviation) [6] 是一种用于高维回归的惩罚方法，主要用于特征选择和参数估计。与传统的 LASSO 方法相比，SCAD 具有更好的无偏性和更强的特征选择能力。SCAD 惩罚函数的导数可以表示为：

$$P'_\lambda(\beta_j) = \begin{cases} \lambda & \text{if } |\beta_j| \leq \lambda \\ \frac{a\lambda - \beta_j}{a-1} & \text{if } \lambda < |\beta_j| \leq a\lambda \\ 0 & \text{if } |\beta_j| > a\lambda \end{cases}$$

其中 λ 是正则化参数，控制惩罚的强度， a 是一个超参数，通常设置为 3.7。

目标函数可以表示为：

$$\min_{\beta} J_n(\beta) + nP_\lambda(\beta) \quad (1)$$

SCAD 的目标函数是非凸的，这对 SCAD 的求解产生一些挑战。下面我们介绍 SCAD 的求解方法。

1.6.1 LQA 算法

LQA 算法的核心思想是通过局部二次近似来优化带有 SCAD 惩罚的目标函数 [6]。它通过将目标函数在当前点附近用二次函数来近似，从而简化优化过程。

1. **初始估计**: 使用普通最小二乘法 (OLS) 计算初始参数估计 $\beta^{(0)}$ 。
2. **局部二次近似**: 对给定的非零 $\beta_j^{(0)}$ ，在其小邻域内，使用以下近似：

$$P_\lambda(|\beta_j|) \approx P_\lambda(|\beta_j^{(0)}|) + \frac{1}{2} \frac{P'_\lambda(|\beta_j^{(0)}|)}{|\beta_j^{(0)}|} (\beta_j^2 - (\beta_j^{(0)})^2)$$

3. **调整的 Newton-Raphson 算法**: 将局部二次近似的惩罚函数代入到目标函数 (1)，并使用 Newton-Raphson 方法更新参数。

$$\beta_1 = \beta^{(0)} - \{\nabla^2 J_n(\beta^{(0)}) + n\Sigma_\lambda(\beta^{(0)})\}^{-1} \{\nabla J_n(\beta^{(0)}) + n\mathbf{U}_\lambda(\beta^{(0)})\}$$

这里 $\mathbf{U}_\lambda(\beta^{(0)}) = \Sigma_\lambda(\beta^{(0)})\beta^{(0)}$ 。

$$\nabla J_n(\beta^{(0)}) = \frac{\partial J_n(\beta^{(0)})}{\partial \beta}, \quad \nabla^2 J_n(\beta^{(0)}) = \frac{\partial^2 J_n(\beta^{(0)})}{\partial \beta \partial \beta^T}$$

$$\Sigma_\lambda(\beta^{(0)}) = \text{diag}\{P'_\lambda(|\beta_1^{(0)}|)/|\beta_1^{(0)}|, \dots, P'_\lambda(|\beta_p^{(0)}|)/|\beta_p^{(0)}|\}$$

4. **迭代**: 重复步骤 2 和步骤 3，直到收敛。若 $|\beta_j| < \eta$ (某个小阈值)，则将该变量剔除。

1.6.2 MM 算法

MM 算法对惩罚函数提出了下面扰动版本的局部二次逼近 [7]，即

$$P_\lambda(|\beta_j|) \approx P_\lambda(|\beta_j^{(0)}|) + \frac{1}{2} \frac{P'_\lambda(|\beta_j^{(0)}|)}{|\beta_j^{(0)}| + \tau_0} (\beta_j^2 - (\beta_j^{(0)})^2)$$

其中 τ_0 是一个非负的扰动参数。

1.6.3 LLA 算法

局部线性逼近 (Local Linear Approximation, LLA) [8] 算法是一种用于优化带有惩罚项的回归模型的方法。以下是 LLA 算法的详细步骤:

1. **初始估计:** 使用普通最小二乘法 (OLS) 计算初始参数估计 $\beta^{(0)}$ 。
2. **局部线性表示:** 在第 k 步, 令 $\beta^{(k)} = (\beta_1^{(k)}, \dots, \beta_p^{(k)})^\top$ 为第 k 步的估计值。对给定的非零 $\beta_j^{(k)}$ 的某个小邻域内, 惩罚函数 $P_\lambda(|\beta_j|)$ 在 $\beta_j^{(k)}$ 的局部线性表示为:

$$P_\lambda(|\beta_j|) \approx P_\lambda(|\beta_j^{(k)}|) + P'_\lambda(|\beta_j^{(k)}|)(|\beta_j| - |\beta_j^{(k)}|)$$

3. **极小化目标函数:** 把惩罚函数的局部线性逼近代入到惩罚最小二乘目标函数中, 并去掉常数项。在 LLA 的帮助下, 极小化下面的目标函数, 得出 $k+1$ 步估计:

$$\beta^{(k+1)} = \arg \min_{\beta} \left\{ J_n(\beta) + n \sum_{j=1}^p P'_\lambda(|\beta_j^{(k)}|) |\beta_j| \right\}$$

4. **迭代直到收敛:** 在步骤 2 和步骤 3 之间进行迭代, 直到收敛。收敛的判定可以基于参数的变化量或者目标函数值的变化量。

2 模拟

2.1 模拟设置

在模拟中, 未知回归系数向量为 $\beta = (-1.5, 1, 1.2, -0.8, 0.6, 0, \dots, 0)^T$, 包含 5 个非零系数, 其他 $p-5$ 个系数为 0。样本量固定为 $n = 300$, 维数 p 变化为 50, 100, 150, 200, 400, 600。自变量 x_i 服从多元正态分布 $N_p(0, \Sigma)$, 其中协方差矩阵 Σ 采用 AR(1) 结构, 即 $\Sigma = (\rho^{|i-j|})_{i,j=1,\dots,p}$, 自相关系数 ρ 取值为 0.3 和 0.6。响应变量 y_i 从伯努利分布生成, 表达式为 $y_i \sim \text{Bernoulli}(\pi(x_i))$, 其中逻辑回归概率为 $\pi(x_i) = 1/(1 + e^{-x_i^T \beta})$ 。模拟重复次数 500。

我们采用了多个准则进行模型比较。首先, 我们计算了非零回归系数被正确估计成非零的平均个数 C , 这一指标用于衡量模型在特征选择方面的准确性, 反映了模型识别真实信号的能力。其次, 我们引入了零回归系数被错误估计成非零的平均个数 IC , 较低的 IC 值表明模型能够有效避免错误识别, 从而提高模型的可靠性。

此外, 我们计算了平均估计误差 EE , 它表示估计的回归系数与真实回归系数之间的 L_2 范数差异, 反映了模型的估计精度。较低的 EE 值表明模型能够准确捕捉真实信号, 具有良好的预测能力。我们还计算了平均错误率 errRate , 分别衡量模型在分类任务中的预测错误比例。由于平均正确率 $\text{accRate} = 1 - \text{errRate}$, 因此与错误率呈负相关关系, 在模拟结果展示中略去这一指标。

最后，我们通过绘制 ROC 曲线并计算 AUC 值来进一步评估模型的分类能力。ROC 曲线展示了不同阈值下的真阳性率与假阳性率之间的关系，而 AUC 值则为 ROC 曲线下的面积，值越接近 1 表示模型的分类性能越优。

2.2 数值结果

数值模拟结果展示在表1中。我们从变量选择和模型性能两个方面进行比较：

- **变量选择**: 指标 C 和 IC 反映了模型的变量选择能力。岭回归方法在选择信号变量方面表现最佳，其 C 值始终保持在 5。然而，相应的 IC 值也始终较高，这表明该方法选择了较多的噪声变量。综合考虑 C 和 IC 两个指标，SCAD 方法表现最优。在高概率选择信号变量的基础上，SCAD 方法能够尽量减少噪声变量的引入。自适应 LASSO 的变量选择效果优于 LASSO，在选择信号变量的能力相似的情况下，自适应 LASSO 更少引入噪声变量。LASSO 的结果与弹性网方法相似。此外，随着维度和相关系数的增大，所有模型的变量选择效果均有所下降。
- **模型性能**: SCAD 方法在 EE 指标上表现突出，显示出其在参数估计上的高准确性。相比之下，Ridge 方法在高维情况下的 EE 值普遍较高，表明其模型拟合效果较差。其他方法的参数估计结果则相对类似。此外，SCAD 方法在多个参数设置中保持了较低的错误率，而 Ridge 方法的错误率则普遍较高。在 AUC 值方面，SCAD 方法在多个设置下均优于其他模型，尤其在 $p = 50$ 时达到了 0.841，表明其具有较强的分类能力。相对而言，Ridge 方法在 AUC 值上的表现最低，尤其在高维情况下（如 $p = 600$ 时，仅为 0.559），这表明其分类能力有限。同样地，随着维度和相关系数的增大，所有模型的分类效果均有所下降。

表 1: Numerical Results for Different Models

rho	p	Method	C(↑)	IC(↓)	EE(↓)	errRate(↓)	AUC(↑)
0.3	50	Lasso	4.970	12.725	0.908	0.253	0.834
		Lasso (ADMM)	4.955	10.070	0.911	0.252	0.835
		Ridge	5.000	45.000	1.218	0.289	0.789
		SCAD	4.910	6.115	0.599	0.243	0.841
		SCAD (LQA)	4.800	0.490	0.594	0.249	0.832
		SCAD (MM)	4.900	1.345	0.597	0.249	0.832
		SCAD(LLA)	4.880	1.095	0.667	0.256	0.822
		Adaptive Lasso	4.950	6.595	0.799	0.251	0.834
		Elastic Net	4.965	12.890	0.910	0.252	0.835

继续下一页

rho	p	Method	C($\uparrow$)	IC($\downarrow$)	EE($\downarrow$)	errRate($\downarrow$)	AUC($\uparrow$)
0.3	100	Lasso	4.950	19.345	1.042	0.252	0.829
		Lasso (ADMM)	4.920	14.225	1.045	0.250	0.830
		Ridge	5.000	95.000	1.623	0.332	0.734
		SCAD	4.900	10.375	0.692	0.242	0.840
		SCAD (LQA)	4.020	0.240	0.803	0.250	0.829
		SCAD (MM)	4.765	3.340	0.826	0.254	0.825
		SCAD(LLA)	4.520	2.400	0.787	0.241	0.840
		Adaptive Lasso	4.935	12.820	1.182	0.258	0.820
		Elastic Net	4.955	19.325	1.040	0.252	0.829
0.3	150	Lasso	4.935	22.690	1.136	0.260	0.821
		Lasso (ADMM)	4.900	15.505	1.138	0.258	0.822
		Ridge	5.000	145.000	1.859	0.366	0.693
		SCAD	4.915	12.640	0.723	0.245	0.835
		SCAD (LQA)	3.605	0.035	0.963	0.258	0.823
		SCAD (MM)	4.695	5.480	1.002	0.260	0.821
		SCAD(LLA)	4.370	1.435	0.782	0.247	0.835
		Adaptive Lasso	4.935	16.435	1.454	0.274	0.805
		Elastic Net	4.945	23.355	1.127	0.259	0.821
0.3	200	Lasso	4.895	26.305	1.184	0.254	0.825
		Lasso (ADMM)	4.830	17.320	1.184	0.253	0.825
		Ridge	5.000	195.000	1.992	0.395	0.668
		SCAD	4.825	14.850	0.808	0.243	0.838
		SCAD (LQA)	3.540	0.010	1.007	0.265	0.817
		SCAD (MM)	4.630	6.850	1.051	0.269	0.813
		SCAD(LLA)	4.245	1.455	0.784	0.246	0.839
		Adaptive Lasso	4.885	18.690	1.657	0.262	0.816
		Elastic Net	4.885	25.595	1.185	0.255	0.825
0.3	400	Lasso	4.740	30.265	1.326	0.276	0.809
		Lasso (ADMM)	4.645	18.040	1.326	0.271	0.810
		Ridge	5.000	395.000	2.283	0.431	0.611
		SCAD	4.770	19.785	0.933	0.254	0.827
		SCAD (LQA)	3.370	0.050	1.122	0.278	0.800
		SCAD (MM)	4.485	12.865	1.162	0.274	0.803
		SCAD(LLA)	3.995	2.120	0.883	0.252	0.827
		Adaptive Lasso	4.655	14.400	1.153	0.288	0.795

继续下一页

rho	p	Method	C($\uparrow$)	IC($\downarrow$)	EE($\downarrow$)	errRate($\downarrow$)	AUC($\uparrow$)
		Elastic Net	4.750	31.300	1.320	0.274	0.810
0.3	600	Lasso	4.645	33.305	1.383	0.273	0.809
		Lasso (ADMM)	4.495	18.230	1.386	0.265	0.809
		Ridge	5.000	595.000	2.300	0.441	0.599
		SCAD	4.695	24.875	0.971	0.256	0.823
		SCAD (LQA)	3.335	0.055	1.192	0.283	0.793
		SCAD (MM)	4.520	18.870	1.234	0.285	0.789
		SCAD(LLA)	3.955	2.745	0.902	0.258	0.822
		Adaptive Lasso	4.560	15.790	1.229	0.283	0.795
		Elastic Net	4.650	32.950	1.389	0.275	0.807
0.6	50	Lasso	4.780	14.150	1.031	0.283	0.795
		Lasso (ADMM)	4.760	11.860	1.033	0.283	0.795
		Ridge	5.000	45.000	1.344	0.318	0.752
		SCAD	4.570	6.410	0.828	0.275	0.804
		SCAD (LQA)	4.775	1.120	0.769	0.282	0.793
		SCAD (MM)	4.795	2.290	0.767	0.282	0.794
		SCAD(LLA)	4.865	2.055	0.880	0.288	0.784
		Adaptive Lasso	4.750	7.130	0.930	0.281	0.795
		Elastic Net	4.790	14.465	1.026	0.282	0.795
0.6	100	Lasso	4.370	18.505	1.296	0.301	0.775
		Lasso (ADMM)	4.305	13.965	1.298	0.296	0.776
		Ridge	5.000	95.000	1.798	0.384	0.668
		SCAD	4.175	9.935	1.023	0.284	0.788
		SCAD (LQA)	4.250	5.125	1.392	0.288	0.786
		SCAD (MM)	4.480	7.580	1.384	0.295	0.779
		SCAD(LLA)	4.520	13.935	2.366	0.283	0.789
		Adaptive Lasso	4.300	11.745	1.253	0.306	0.767
		Elastic Net	4.390	18.280	1.299	0.301	0.774
0.6	150	Lasso	4.195	21.810	1.384	0.307	0.768
		Lasso (ADMM)	4.105	15.065	1.385	0.302	0.768
		Ridge	5.000	145.000	2.022	0.413	0.632
		SCAD	4.165	12.055	1.076	0.294	0.780
		SCAD (LQA)	2.860	0.060	1.256	0.306	0.772
		SCAD (MM)	3.720	7.070	1.272	0.308	0.766
		SCAD(LLA)	3.245	0.510	1.122	0.292	0.783

继续下一页

rho	p	Method	C(↑)	IC(↓)	EE(↓)	errRate(↓)	AUC(↑)
		Adaptive Lasso	4.135	14.770	1.477	0.315	0.756
		Elastic Net	4.240	22.350	1.371	0.307	0.768
0.6	200	Lasso	4.060	22.075	1.457	0.316	0.758
		Lasso (ADMM)	3.920	14.035	1.459	0.309	0.761
		Ridge	5.000	195.000	2.166	0.439	0.606
		SCAD	3.905	12.840	1.191	0.299	0.775
		SCAD (LQA)	2.665	0.020	1.299	0.320	0.763
		SCAD (MM)	3.580	9.300	1.299	0.316	0.752
		SCAD(LLA)	3.065	0.530	1.180	0.293	0.783
		Adaptive Lasso	4.000	15.835	1.540	0.319	0.751
		Elastic Net	4.065	22.580	1.453	0.318	0.759
0.6	400	Lasso	3.645	26.275	1.610	0.330	0.742
		Lasso (ADMM)	3.525	14.470	1.612	0.323	0.746
		Ridge	5.000	395.000	2.325	0.454	0.573
		SCAD	3.655	17.990	1.319	0.307	0.762
		SCAD (LQA)	2.390	0.165	1.445	0.334	0.737
		SCAD (MM)	3.330	15.880	1.462	0.345	0.718
		SCAD(LLA)	2.695	0.985	1.315	0.301	0.772
		Adaptive Lasso	3.535	15.375	1.467	0.337	0.728
		Elastic Net	3.645	26.850	1.609	0.328	0.743
0.6	600	Lasso	3.560	27.350	1.659	0.331	0.736
		Lasso (ADMM)	3.400	13.505	1.659	0.323	0.740
		Ridge	5.000	595.000	2.342	0.470	0.559
		SCAD	3.550	19.725	1.384	0.314	0.757
		SCAD (LQA)	2.360	0.145	1.460	0.356	0.728
		SCAD (MM)	3.260	18.775	1.488	0.368	0.710
		SCAD(LLA)	2.670	1.840	1.344	0.307	0.765
		Adaptive Lasso	3.445	15.445	1.556	0.341	0.721
		Elastic Net	3.590	26.760	1.660	0.332	0.736

2.3 ROC 曲线图

模型在不同设置下的 ROC 曲线图如图 1 至 12 所示。ROC (Receiver Operating Characteristic) 曲线是一种用于评估二分类模型性能的工具，广泛应用于医学、机器学习和统计学等领域。它通过画出不同阈值下的真正率 (True Positive Rate, TPR) 与假正率 (False Positive Rate, FPR) 之间的关系，帮助我们理解模型在不同决策阈值下的

表现。ROC 曲线图的坐标系是一个二维平面，X 轴表示假正率 FPR，Y 轴表示真正率 TPR。通常，曲线越靠近左上角，模型性能越好。

ROC 曲线图的分析结果与 2.2 中的结论相一致。具体而言，Ridge 方法的分类能力在所有模型中表现最差，ROC 曲线的形状显示其在高假正率下的真正率提升有限，表明该模型在区分正负类样本时存在显著的不足。相比之下，SCAD 方法的分类能力则表现最佳，其 ROC 曲线在整个假正率范围内均保持较高的真正率，尤其在低假正率区域，SCAD 方法能够有效地识别正类样本，尤其在高维情况下，展现出优越的分类性能。而其他方法的 ROC 曲线相对较为集中，表明它们的分类能力相似。

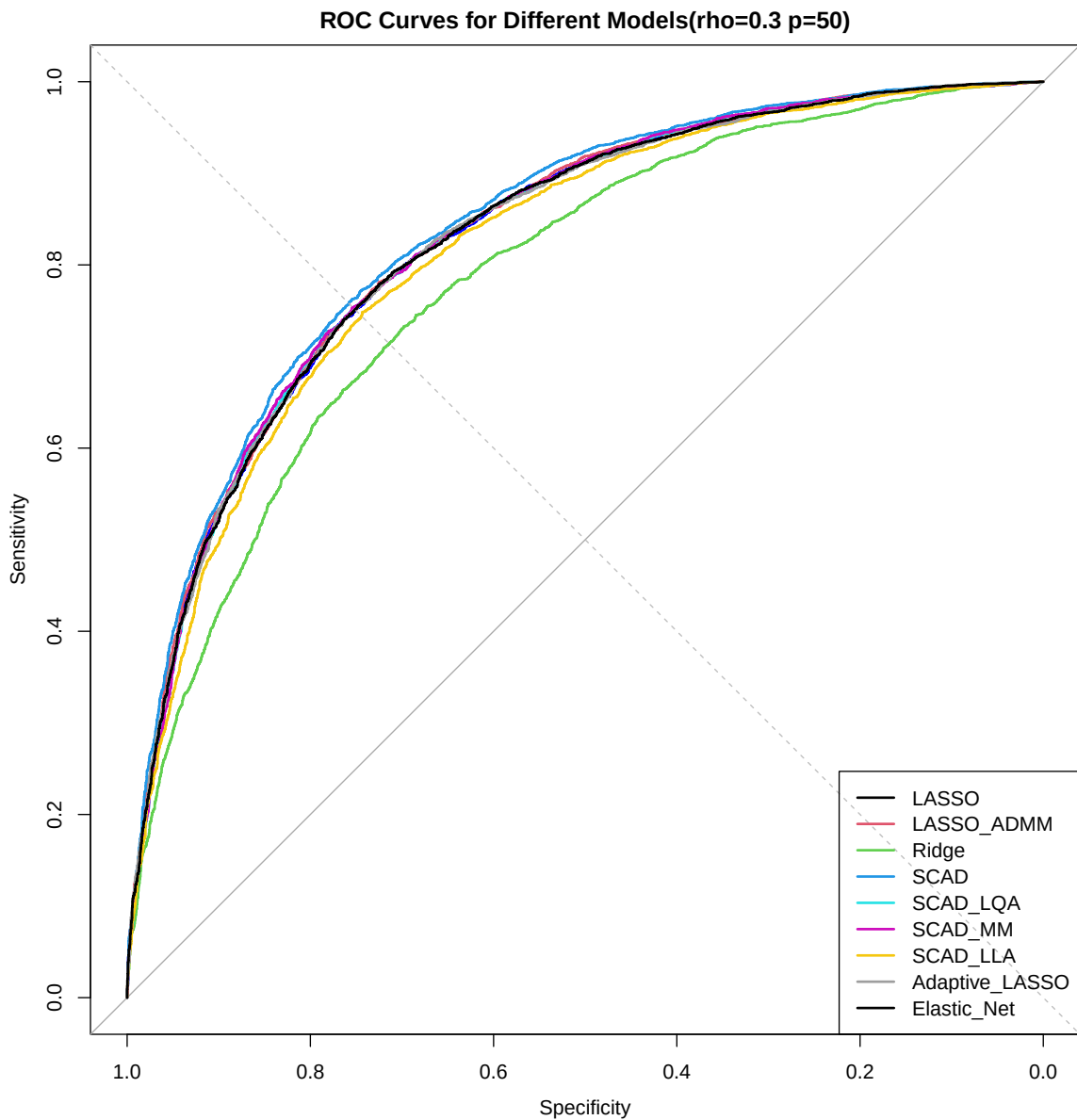


图 1: ROC Curves for Different Models ($\rho=0.3$, $p=50$)

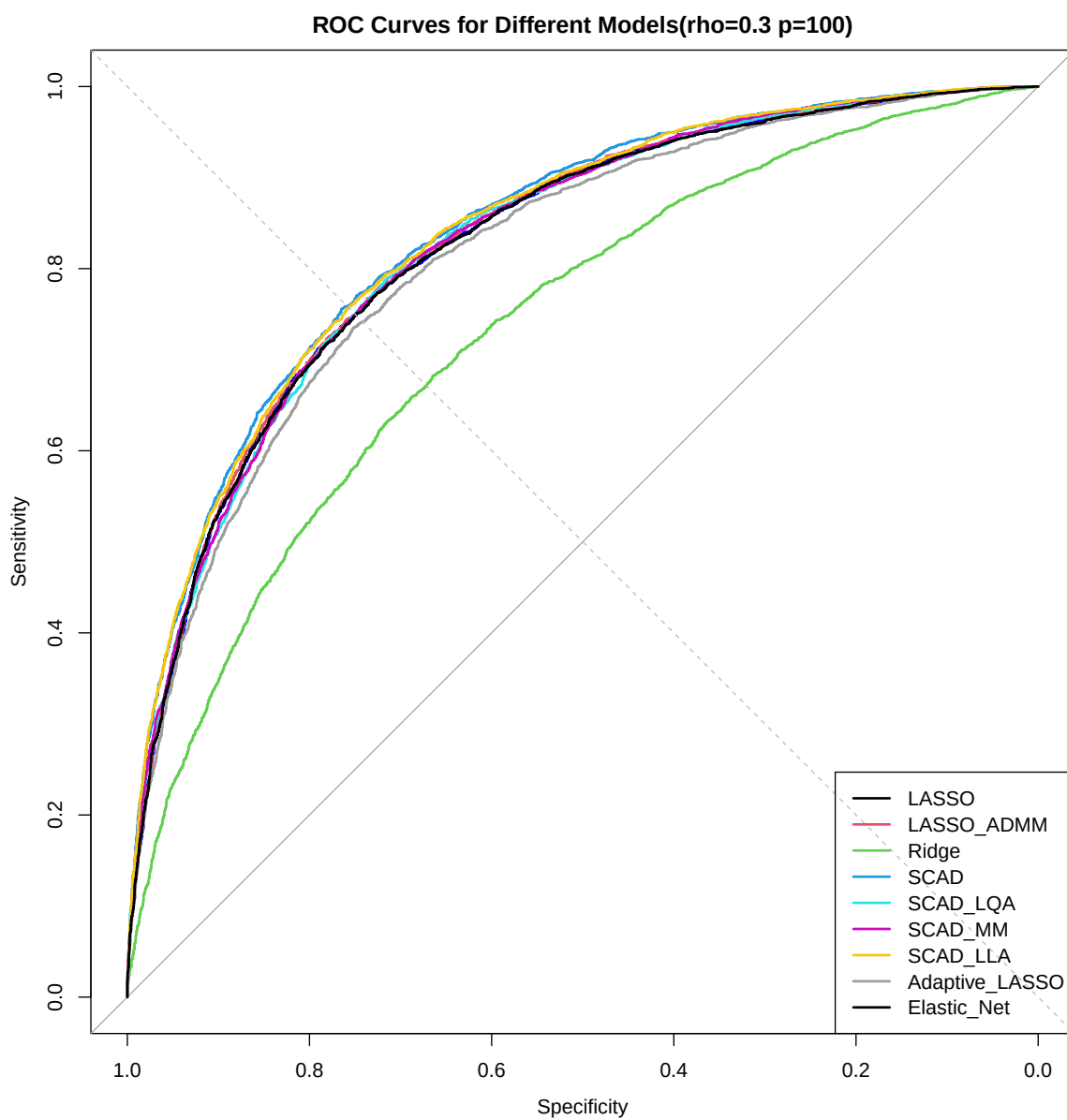


图 2: ROC Curves for Different Models ($\rho=0.3$, $p=100$)

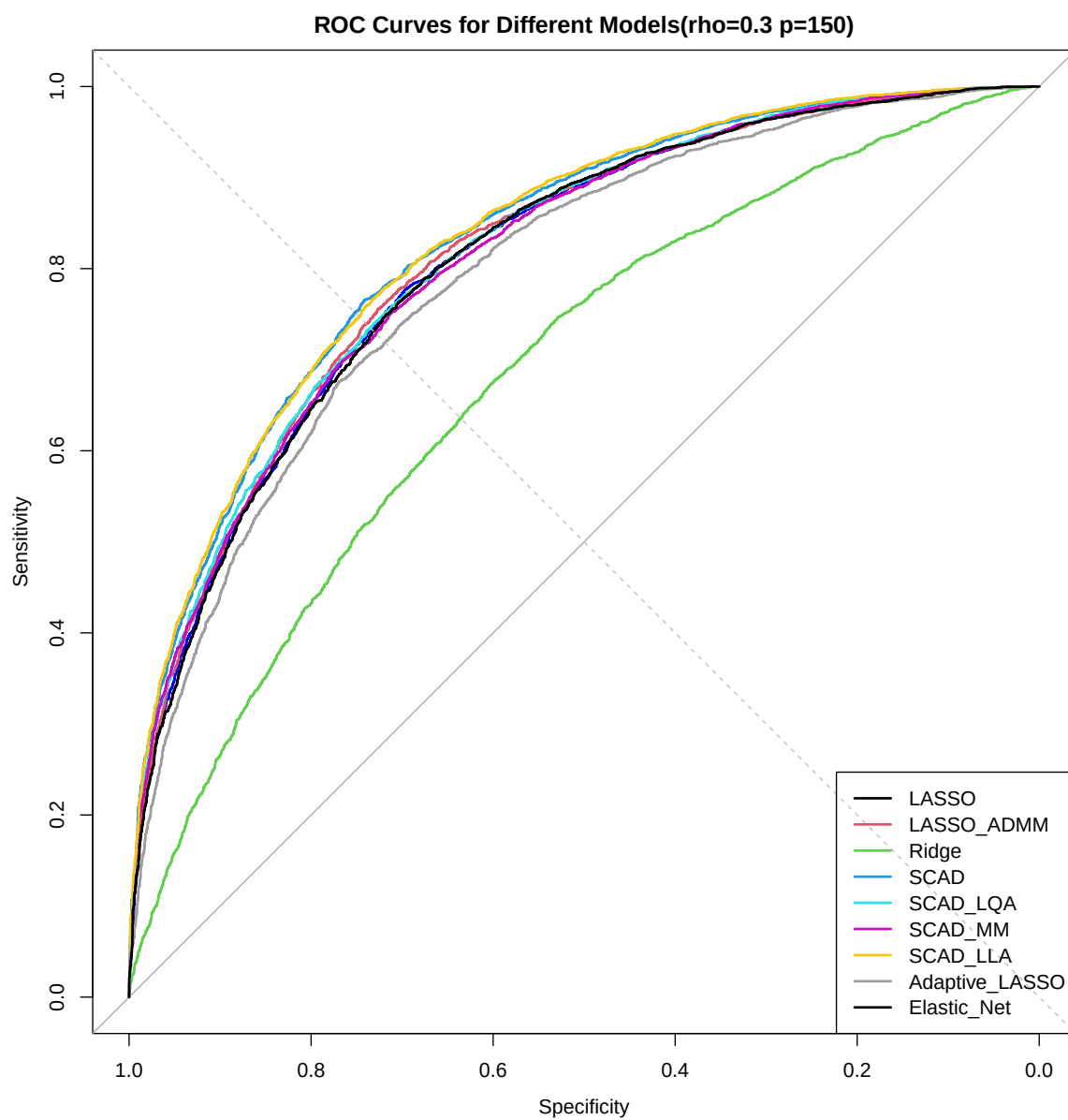


图 3: ROC Curves for Different Models ($\rho=0.3$, $p=150$)

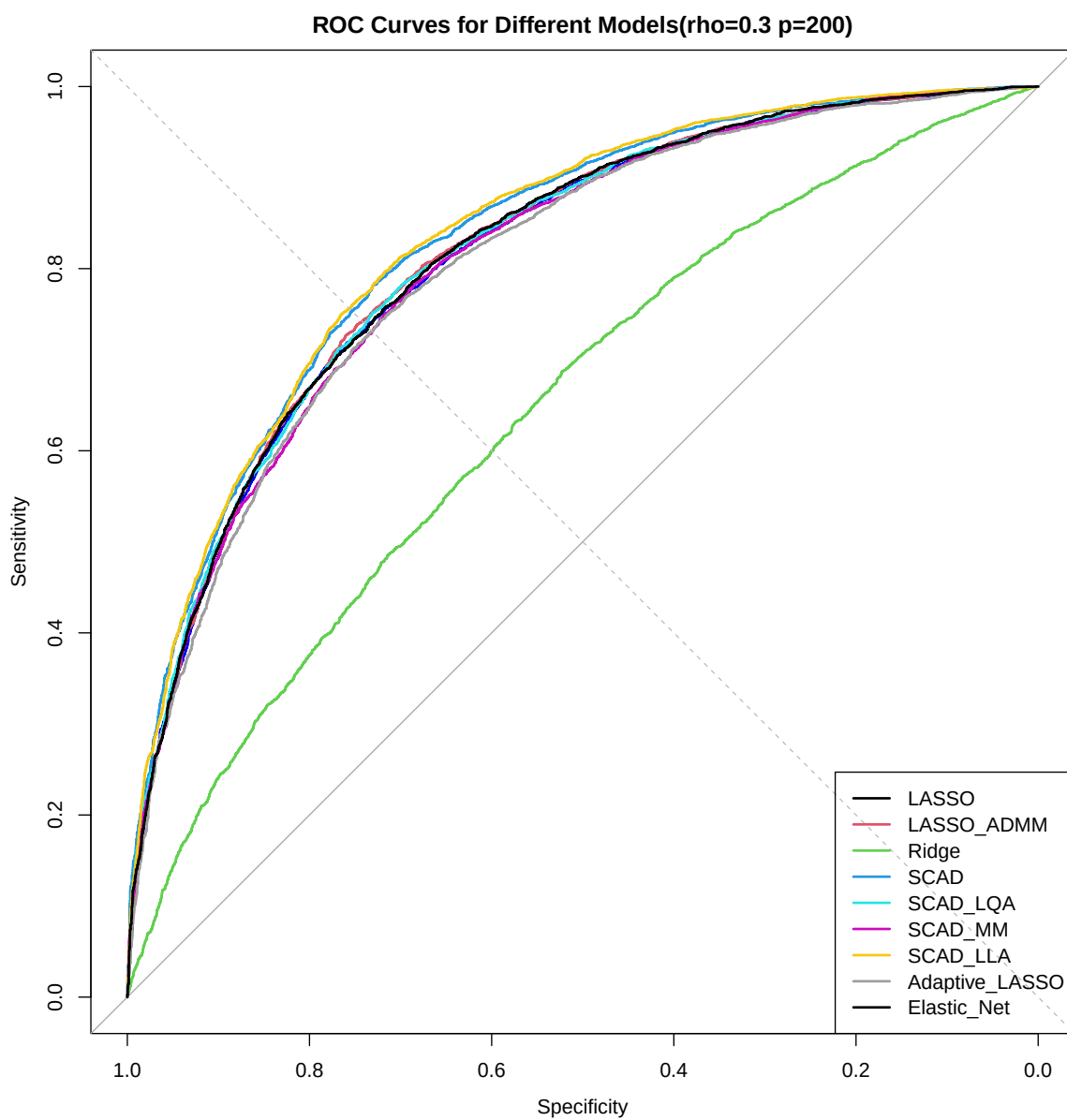


图 4: ROC Curves for Different Models ($\rho=0.3$, $p=200$)

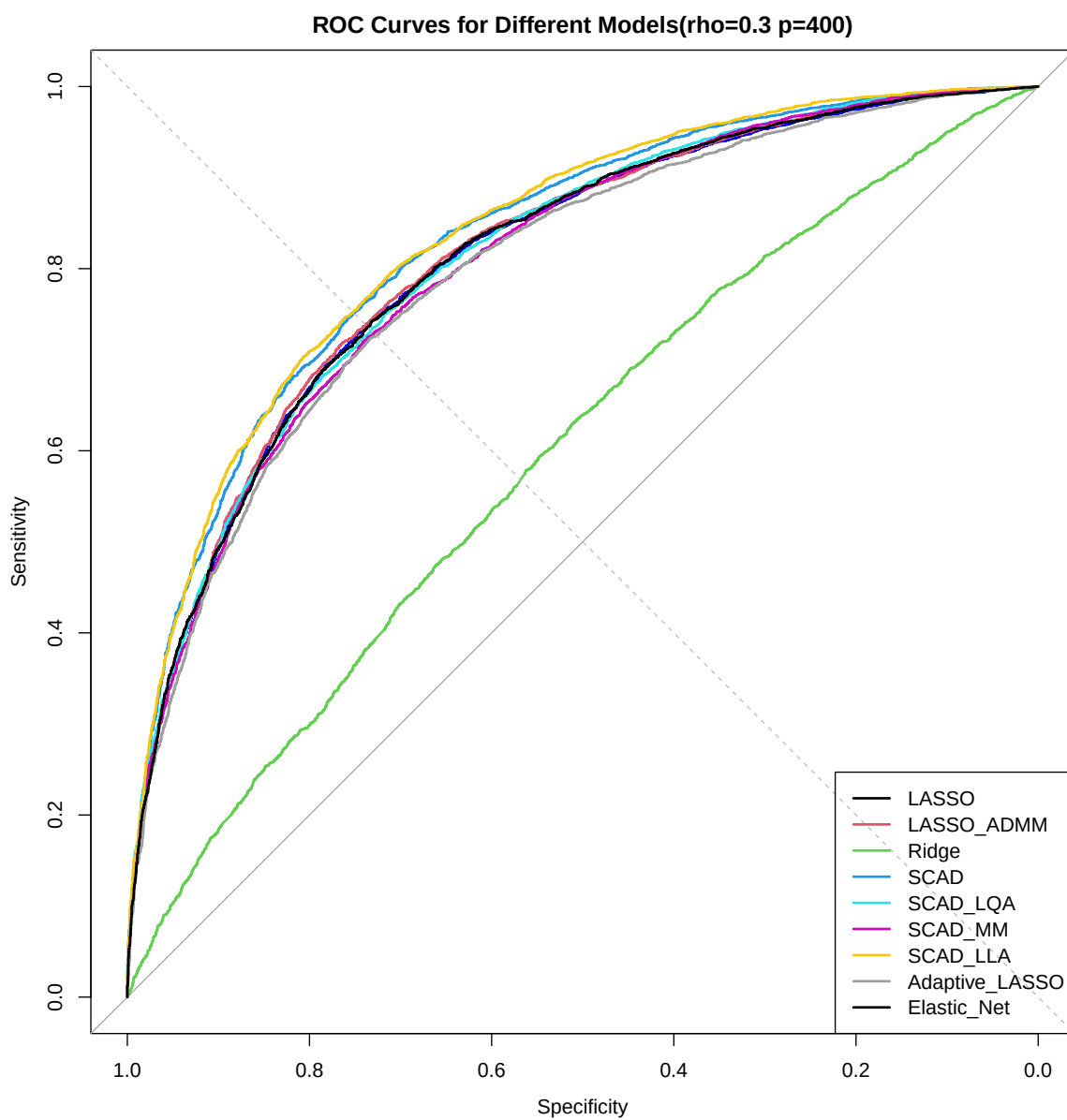


图 5: ROC Curves for Different Models ($\rho=0.3$, $p=400$)

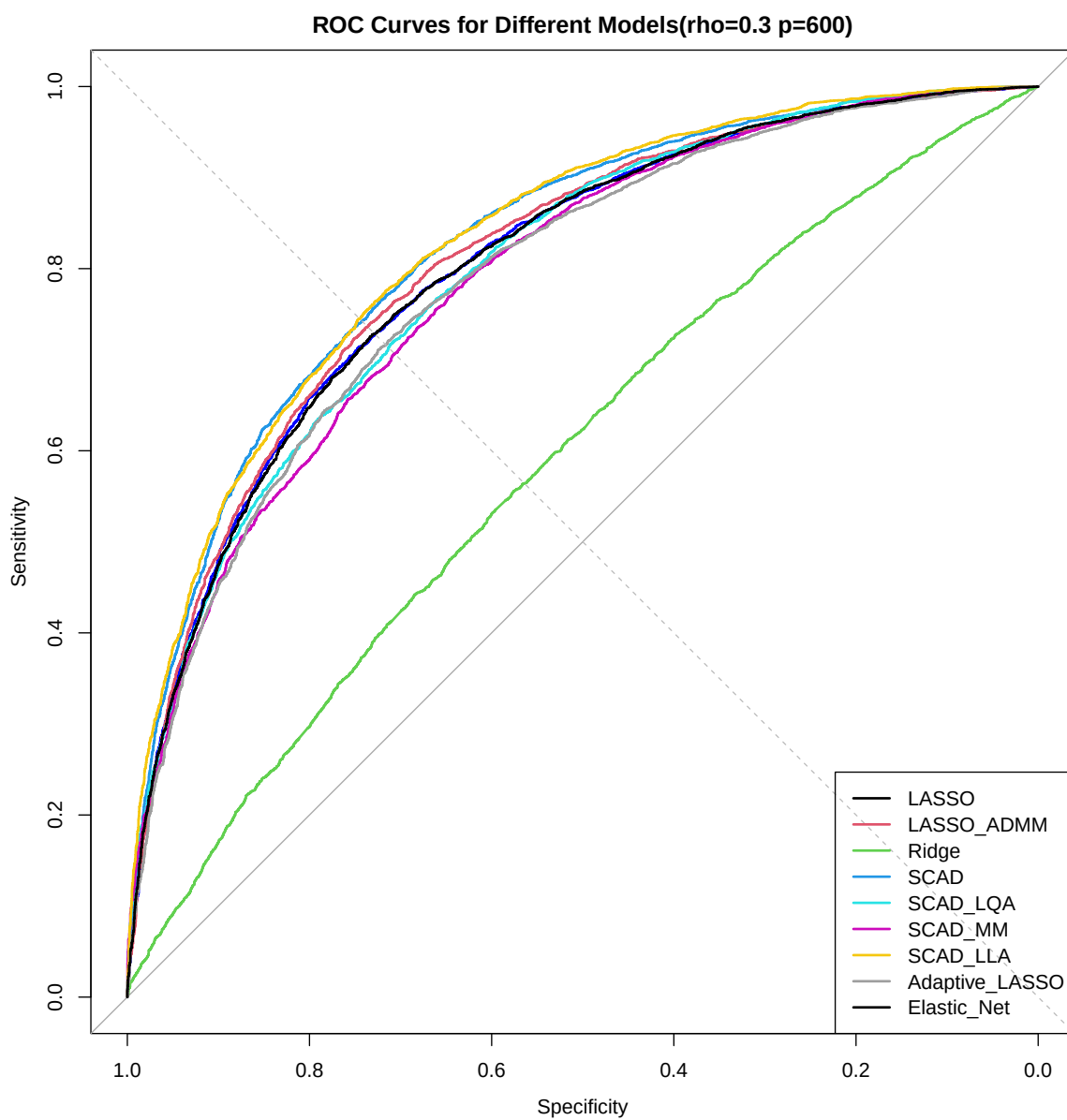


图 6: ROC Curves for Different Models ($\rho=0.3$, $p=600$)

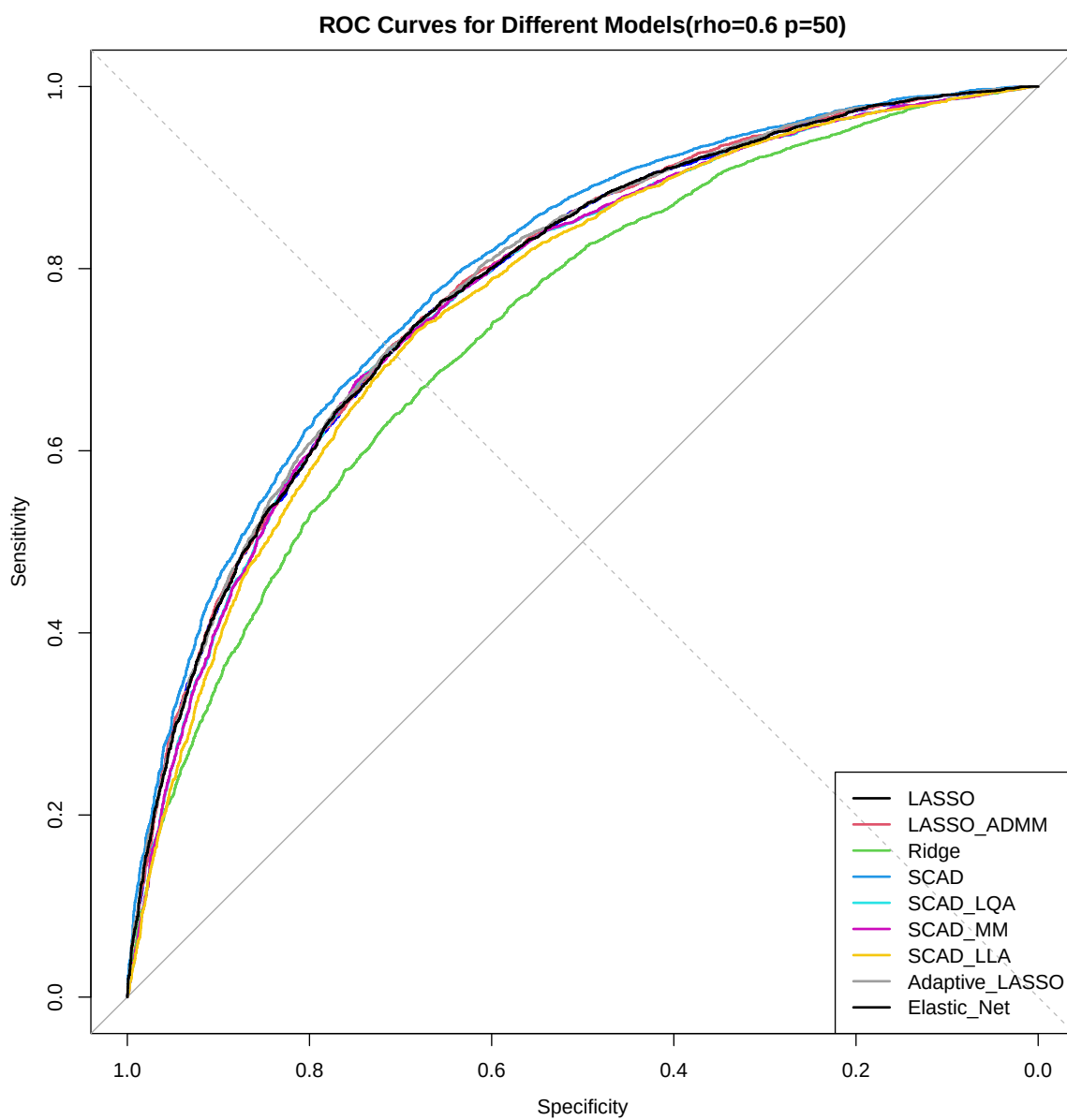


图 7: ROC Curves for Different Models ($\rho=0.6$, $p=50$)

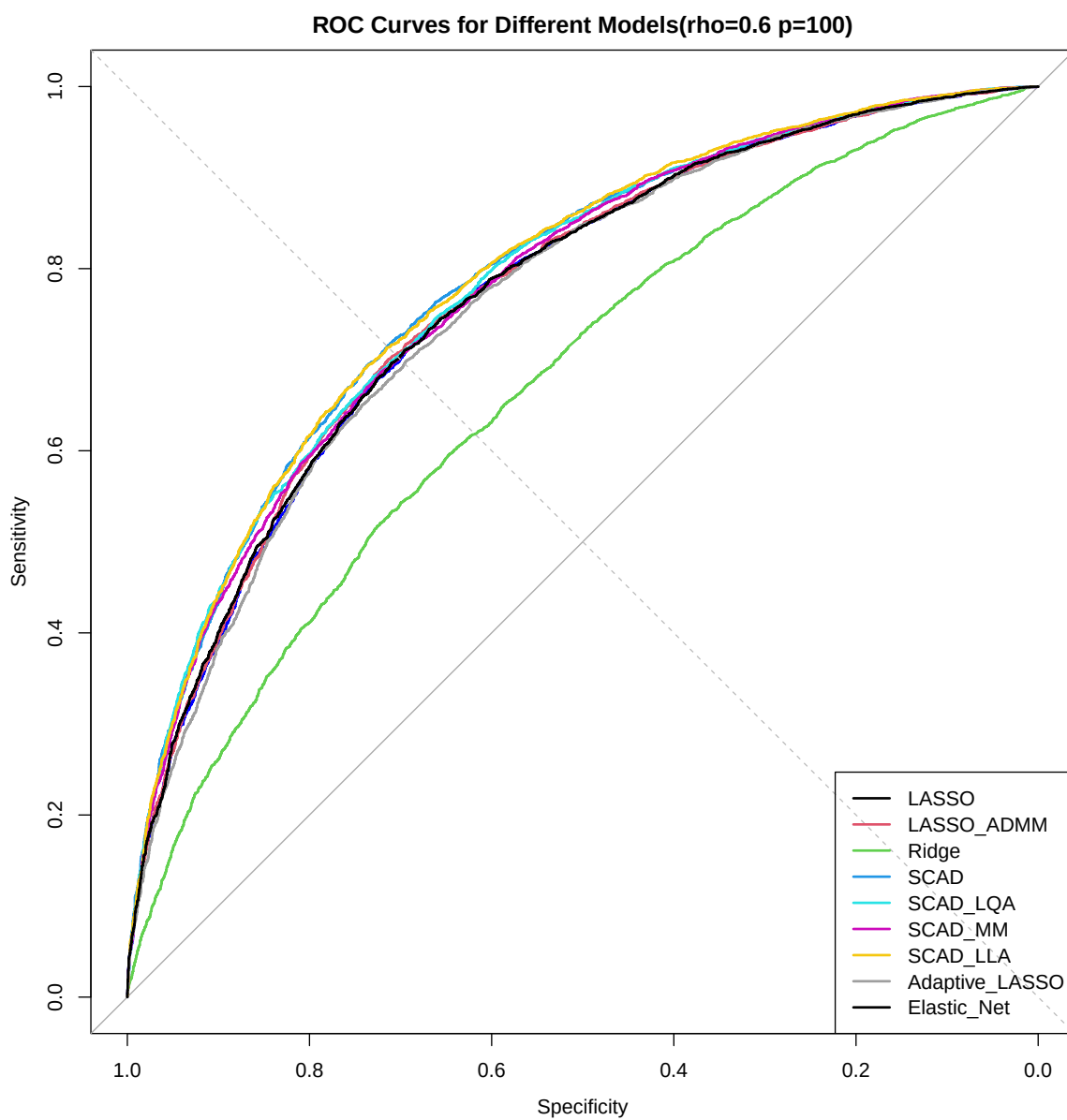


图 8: ROC Curves for Different Models ($\rho=0.6$, $p=100$)

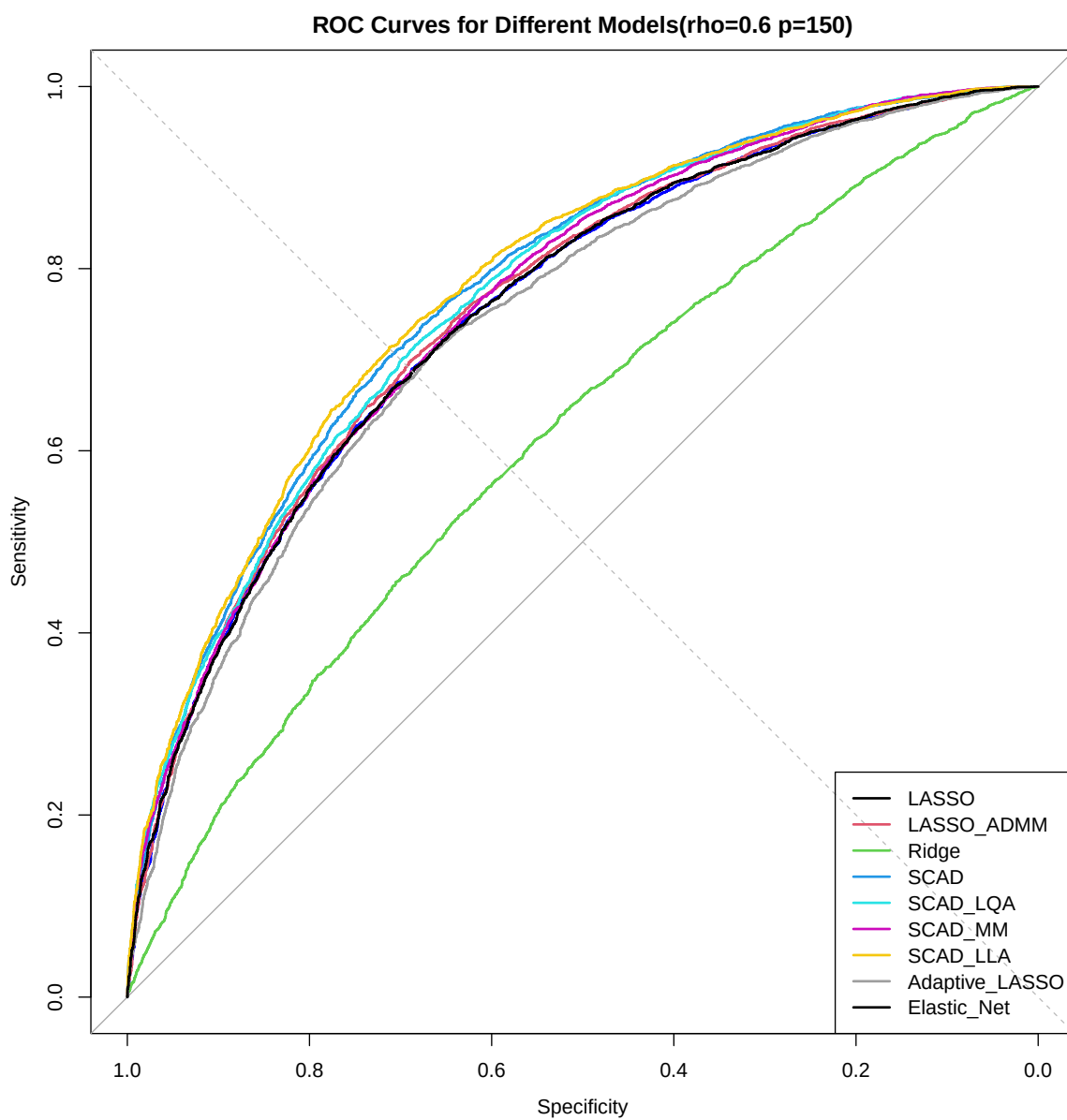


图 9: ROC Curves for Different Models ($\rho=0.6$, $p=150$)

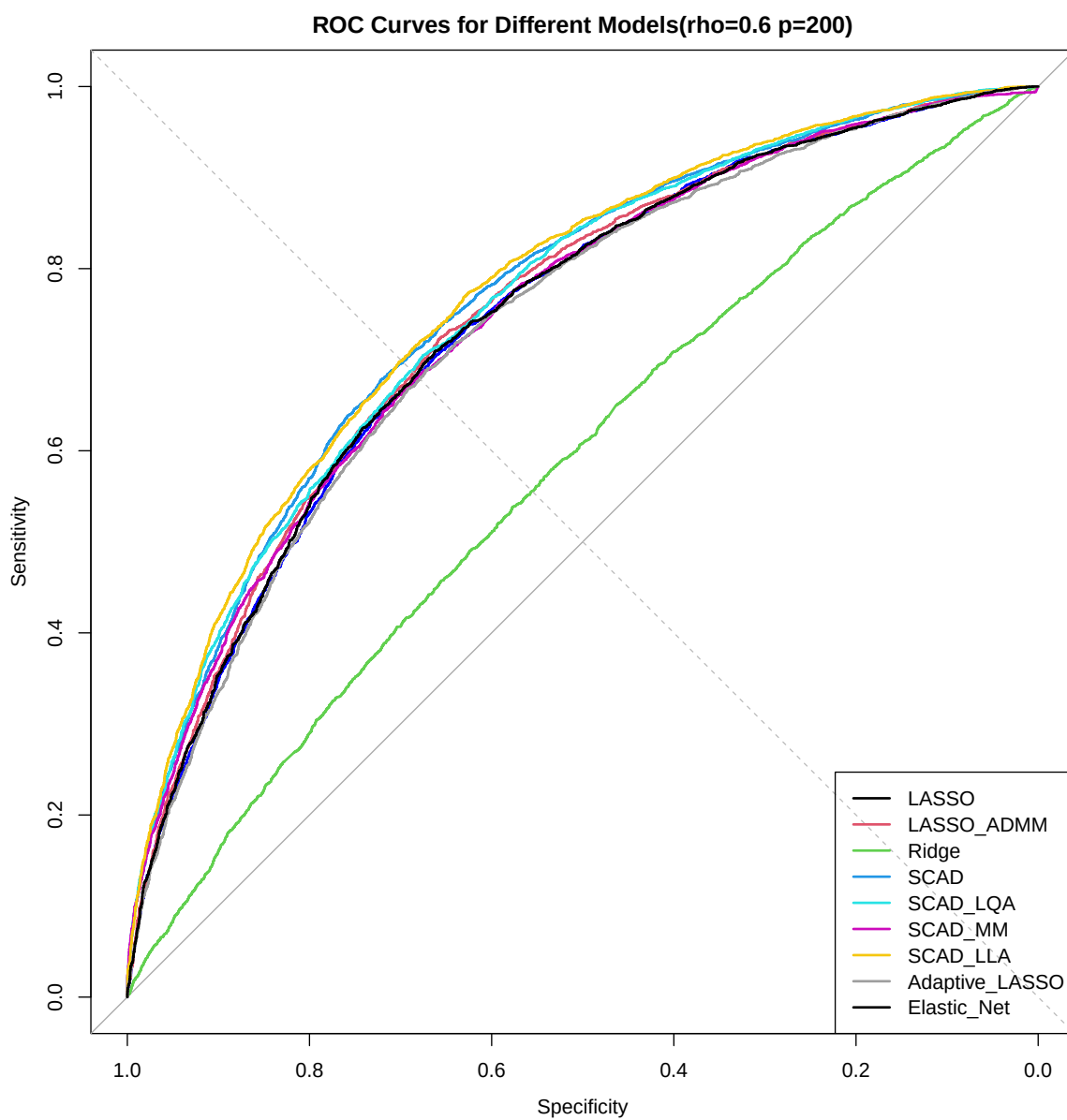


图 10: ROC Curves for Different Models ($\rho=0.6$, $p=200$)

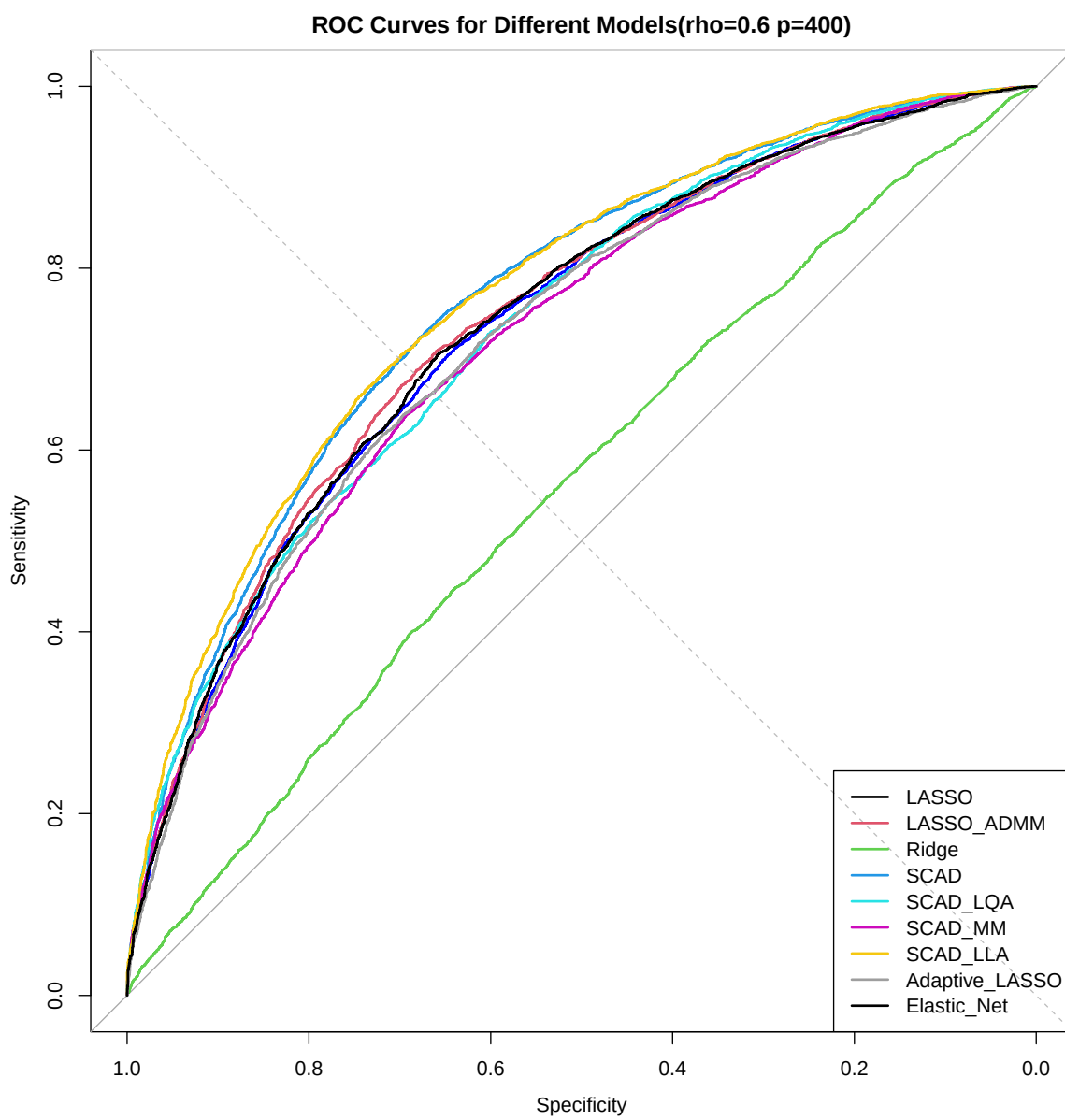


图 11: ROC Curves for Different Models ($\rho=0.6$, $p=400$)

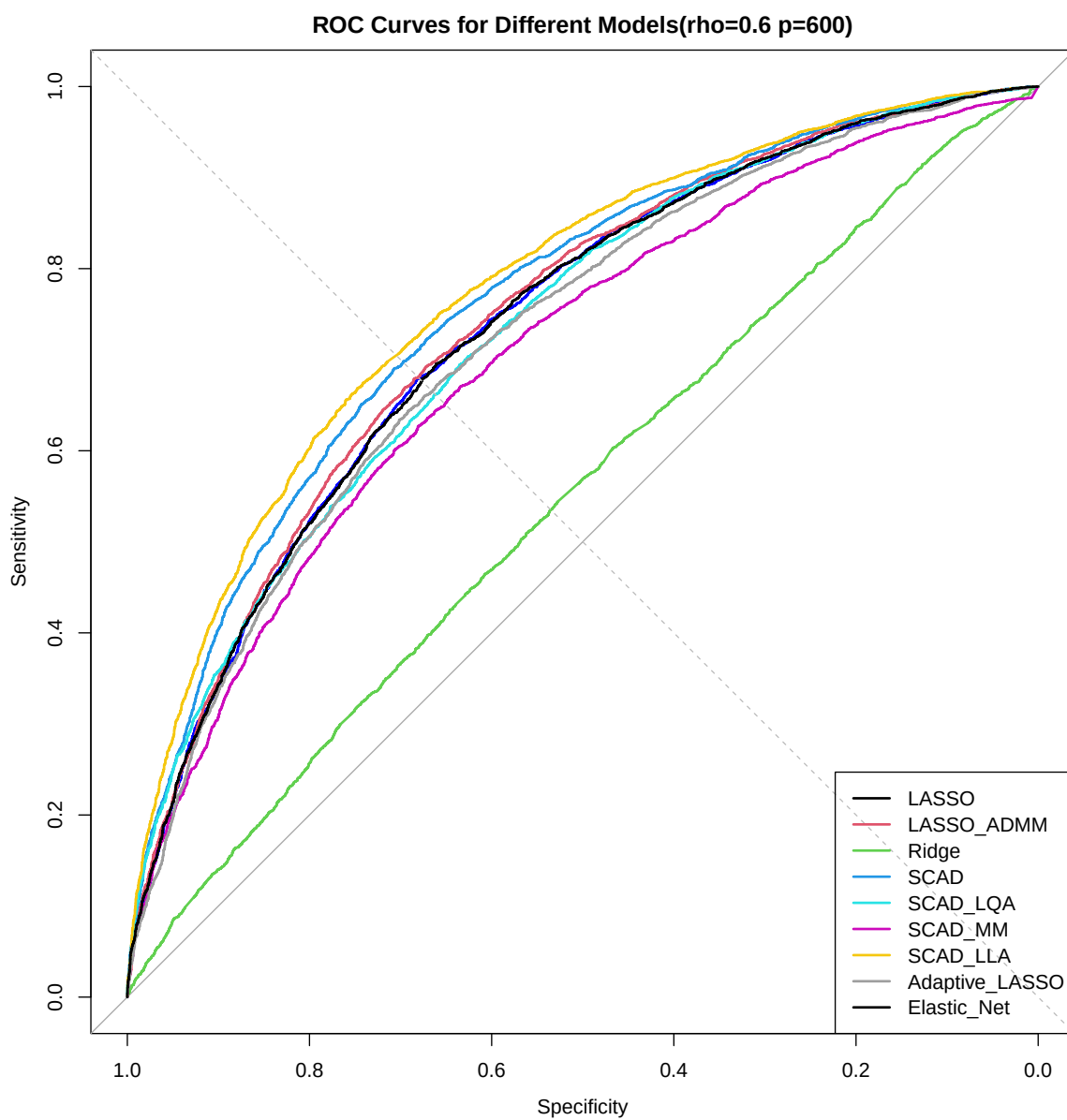


图 12: ROC Curves for Different Models ($\rho=0.6$, $p=600$)

参考文献

- [1] David R Cox. The regression analysis of binary sequences. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 20(2):215–232, 1958.
- [2] McCullagh and Nelder. Generalized linear models, 1989.
- [3] Robert Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 58(1):267–288, 1996.
- [4] Hui Zou. The adaptive lasso and its oracle properties. *Journal of the American statistical association*, 101(476):1418–1429, 2006.
- [5] Hui Zou and Trevor Hastie. Regularization and variable selection via the elastic net. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 67(2):301–320, 2005.
- [6] Jianqing Fan and Runze Li. Variable selection via nonconcave penalized likelihood and its oracle properties. *Journal of the American statistical Association*, 96(456):1348–1360, 2001.
- [7] David R Hunter and Runze Li. Variable selection using mm algorithms. *Annals of statistics*, 33(4):1617, 2005.
- [8] Hui Zou and Runze Li. One-step sparse estimates in nonconcave penalized likelihood models. *Annals of statistics*, 36(4):1509, 2008.